ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH

TRƯỜNG ĐẠI HỌC BÁCH KHOA

BÁO CÁO

BÀI TẬP LỚN MÔN KIẾN TRÚC MÁY TÍNH

Milestone 1

Design of a Vending Machine

Giảng viên hướng dẫn: Trần Hoàng Linh

Họ và tên: Võ Phan Tiến Đạt

MSSV: 2310716

MILESTONE 1: Design of a Vending Machine

I. Phương pháp giải

Vending Machine is a dispenser machine that receives coins or bills and dispenses soft drinks or snacks. A simple vending machine controller is described in Figure 1.

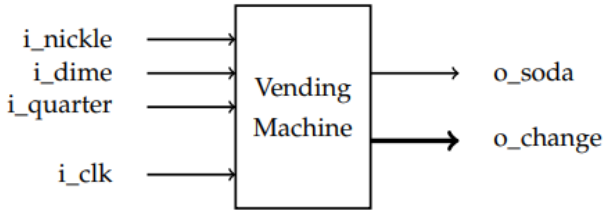


Figure 1: Vending machine's ports

- Ta sẽ thiết kế máy trạng thái theo kiểu MEALY
- Có 3 ngõ vào: i_nickle (5 đồng); i_dime (10 đồng); i_quarter (25 đồng) lần lượt là X2 X1 X0
- Có 2 ngõ ra: o_soda (1 bit Z3) và o_change (3 bit Z2 Z1 Z0 vì tiền thừa dạng 3 bit data: 000 = ¢0; 001 = ¢5; 010 = ¢10; 011 = ¢15; 100 = ¢20)
- Cần 2 bit trạng thái Q1Q0 vì có 4 trạng thái. Lưu ý trạng thái là số tiên hiện có trong máy khi chưa đủ 20 đồng = 1 lon soda → nó là số tiền máy đang tạm giữ của khách hàng cho giao dịch hiện tại.
- Ta sẽ gán trạng thái 0 đồng = 00, 5 đồng = 01, 10 đồng = 10, 15 đồng = 11.
- Ví dụ trong dạng sóng, tại 1: đưa vào 10 đồng, nên ngõ vào i_dime bật lên trong 1 chu kỳ, tiếp theo đưa vào 25 đồng nên ngõ vào i_quarter bật lên trong 1 chu kỳ. tại chu kỳ tiếp theo, vì tổng tiền đưa vào vượt quá 25 đồng nên máy trả 1 lon soda và tiền thừa dạng 011 = 15 đồng.
- Ví dụ nhét 25 đồng vào, máy sẽ trả 1 lon soda và 5 đồng thừa (Z2Z1Z0 = 001). Sau khi hoàn tất một giao dịch (nhả soda và thối tiền), máy phải **reset** về trạng thái ban đầu để sẵn sàng cho người mua tiếp theo. Nên khi đã thối tiền thì trạng thái Q1+ và Q0+ phải là 00. Khi nhét 10 đồng vào (chưa đủ 1 lon soda) thì máy sẽ giữ lại 10 đồng nên Z2Z1Z0 = 000 và biến trạng thái Q1+Q0+ là 10.
- Bảng trạng thái:

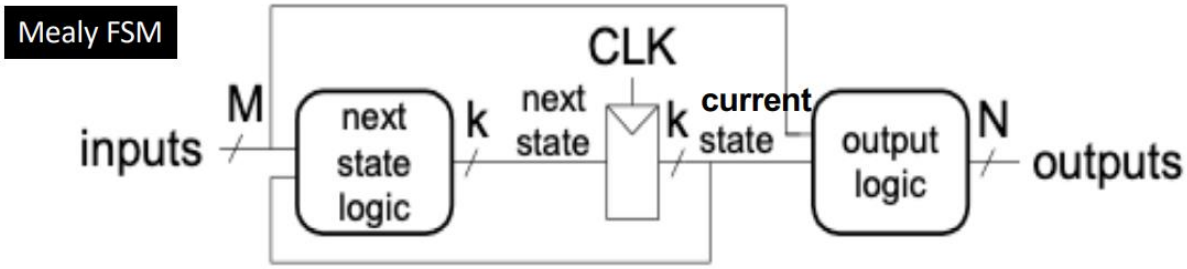
- **Lưu ý rằng mỗi lần chỉ được cho vào 1 đồng xu**

Trạng thái HT		**Ngõ vào**			**Ngõ ra**				**Trạng thái kế tiếp**		**D FlipFlop (D = Q+)**	
Q1	Q0	X2	X1	X0	Z3 (soda)	Z2	Z1	Z0	Q1+	Q0+	D1	D0
0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	1	1	0	0	1	0	0	0	0
0	0	0	1	0	0	0	0	0	1	0	1	0
0	0	1	0	0	0	0	0	0	0	1	0	1
0	1	0	0	0	0	0	0	0	0	1	0	1
0	1	0	0	1	1	0	1	0	0	0	0	0
0	1	0	1	0	0	0	0	0	1	1	1	1
0	1	1	0	0	0	0	0	0	1	0	1	0
1	0	0	0	0	0	0	0	0	1	0	1	0
1	0	0	0	1	1	0	1	1	0	0	0	0
1	0	0	1	0	1	0	0	0	0	0	0	0
1	0	1	0	0	0	0	0	0	1	1	1	1
1	1	0	0	0	0	0	0	0	1	1	1	1
1	1	0	0	1	1	1	0	0	0	0	0	0
1	1	0	1	0	1	0	0	1	0	0	0	0
1	1	1	0	0	1	0	0	0	0	0	0	0

- Từ bảng trạng thái ta tìm được:

- D1 = Q1'.Q0. X2.X1'.X0' + Q1'.X2'.X1.X0' + Q1.X2'.X1'.X0' + Q1.Q0'.X1'.X0'
- D0 = Q0'.X2.X1'.X0' + Q1'.Q0.X2'.X0' + Q0.X2'.X1'.X0'
- Z3 = o_soda = Q1.Q0.X2.X1'.X0' + Q1.X2'.X1.X0' + X2'.X1'.X0
- Z2 = Q1.Q0. X2'.X1'.X0
- Z1 = Q1'.Q0. X2'.X1'.X0 + Q1.Q0'. X2'.X1'.X0
- Z0 = Q1.Q0. X2'.X1.X0' + Q0'. X2'.X1'.X0

- Ta sẽ thiết kế code SystemVerilog dựa trên sơ đồ này:



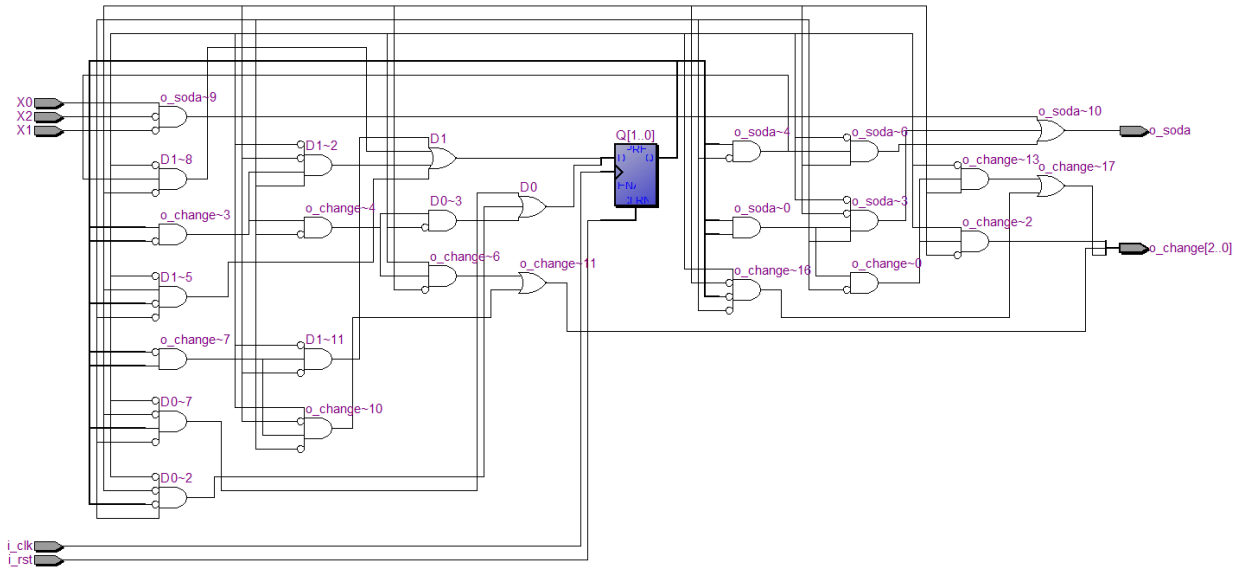
II. Chương trình SystemVerilog

- Chương trình SystemVerilog

```
module vending_machine (
  input logic i_clk,
  input logic i_rst,
  input logic X2,      // X2 là i_nikel
  input logic X1,      // X1 là i_dime
  input logic X0,      // X0 là i_quarter
  output logic o_soda,
  output logic [2:0] o_change
        );

  logic Q1, Q0;      // trạng thái hiện tại
  logic D1, D0;      // Trạnh thái kế tiếp, cũng là ngõ vào của DFF

// Logic tuần tự: thiết kế 2 D-Flip-Flop
   always_ff @(posedge i_clk or posedge i_rst) begin
   if (i_rst == 1'b1) begin
      Q1 <= 1'b0;
      Q0 <= 1'b0;
   end
   else begin
      Q1 <= D1;           // Q1 = D1
      Q0 <= D0;           // Q0 = D0
  end
  end

// dùng always_comb để tạo khối next state logic
always_comb begin

  D1 = (~Q1 & Q0 & X2 & ~X1 & ~X0) | (~Q1 & ~X2 & X1 & ~X0) | (Q1 & ~X2 &
~X1 & ~X0) | (Q1 & ~Q0 & ~X1 & ~X0) ;

   D0 = (~Q0 & X2 & ~X1 & ~X0) | (~Q1 & Q0 & ~X2 & ~X0) | (Q0 & ~X2 & ~X1
& ~X0);
end

// dùng always_comb để tạo khối output logic
always_comb begin
  o_soda     = (Q1 & Q0 & X2 & ~X1 & ~X0) | (Q1 & ~X2 & X1 & ~X0) | (~X2 &
~X1 & X0);
  o_change[2] = (Q1 & Q0 & ~X2 & ~X1 & X0);
```

```
  o_change[1] = (~Q1 & Q0 & ~X2 & ~X1 & X0) | (Q1 & ~Q0 & ~X2 & ~X1 & X0);
  o_change[0] = (Q1 & Q0 & ~X2 & X1 & ~X0) | (~Q0 & ~X2 & ~X1 & X0);
end
  endmodule: vending_machine
```

- **Dạng RTL**



III. Kiểm tra dạng sóng bằng EDA

```
`timescale 1ns/1ps
module testbench;

   // Khai báo các tín hiệu để kết nối với module máy bán hàng
   logic      i_clk;
   logic      i_rst;
   logic      X2, X1, X0; // nickel, dime, quarter
   logic      o_soda;
   logic [2:0]  o_change;

   // Gọi module thiết kế của mày vào (DUT) và nối dây
   vending_machine dut (
      .i_clk(i_clk),
      .i_rst(i_rst),
```

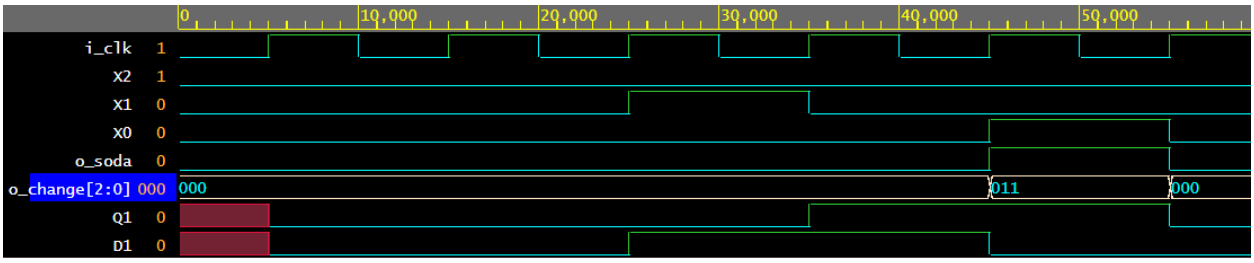
```
        .X2(X2),
        .X1(X1),
        .X0(X0),
        .o_soda(o_soda),
        .o_change(o_change)
    );

    // Tạo xung clock (chu kỳ 10ns)
// forever để Bắt đầu một vòng lặp chạy mãi mãi, không dừng
// #5 để yêu cầu bộ mô phỏng đợi trong 5 đơn vị thời gian.
// Nếu timescale là 1ns/1ps thì #5 nghĩa là đợi 5ns
// i_clk = ~i_clk;: Sau khi đợi xong, lấy giá trị đảo của i_clk gán ngược lại cho chính
nó.
    initial begin
        i_clk = 0;
        forever #5 i_clk = ~i_clk;     //
    end

    // Viết kịch bản mô phỏng để tạo tín hiệu vào
    initial begin
        // Lệnh để EDA Playground tạo và mở file sóng
        $dumpfile("dump.vcd");
        $dumpvars(0, testbench);

        // reset mạch
        i_rst = 1;
        {X2, X1, X0} = 3'b000;
        repeat(2) @(posedge i_clk);
        i_rst = 0;


        // KỊCH BẢN 1: 10c + 25c --> 1 lon soda và 15c tiền thừa
        // Bỏ vào 1 Dime
        @(posedge i_clk);    // Đợi đến cạnh lên của clock chu kỳ N
        X1 <= 1;             // X1 lên mức 1. Tín hiệu X1 sẽ giữ mức 1 này
        @(posedge i_clk);    // Đợi đến cạnh lên của clock chu kỳ N+1
        X1 <= 0;             // ra lệnh cho X1 xuống mức 0.,
       @(posedge i_clk);    // Nếu chỉ set X1=1 và để đó thì tín hiệu X1 sẽ giữ nguyên ở
mức 1 mãi mãi.
        // Bỏ vào 1 Quarter
        X0 <= 1;
        @(posedge i_clk);
        X0 <= 0;
```

```
    // Chờ 2 chu kỳ để quan sát ngõ ra
  repeat(2) @(posedge i_clk);


    // KỊCH BẢN 2: 5c + 10c + 5c --> 1 lon soda và 0 có tiền thừa
    // Bỏ vào 5c
    X2 <= 1; @(posedge i_clk); X2 <= 0;
    // Bỏ vào 10c
    @(posedge i_clk);
    X1 <= 1; @(posedge i_clk); X1 <= 0;
    // Bỏ vào 5c
    @(posedge i_clk);
    X2 <= 1; @(posedge i_clk); X2 <= 0;

  repeat(2) @(posedge i_clk);


    // KỊCH BẢN 3: 5c + 25c --> 1 lon soda và 10c tiền thừa
    // Bỏ vào 1 nickel
    @(posedge i_clk);
    X2 <= 1;
    @(posedge i_clk);
    X2 <= 0;
   @(posedge i_clk);
    // Bỏ vào 1 Quarter
    X0 <= 1;
    @(posedge i_clk);
    X0 <= 0;

    // Chờ 2 chu kỳ để quan sát ngõ ra
  repeat(2) @(posedge i_clk);

    $finish; // Dừng mô phỏng
  end

endmodule
```

- **Kết quả:**

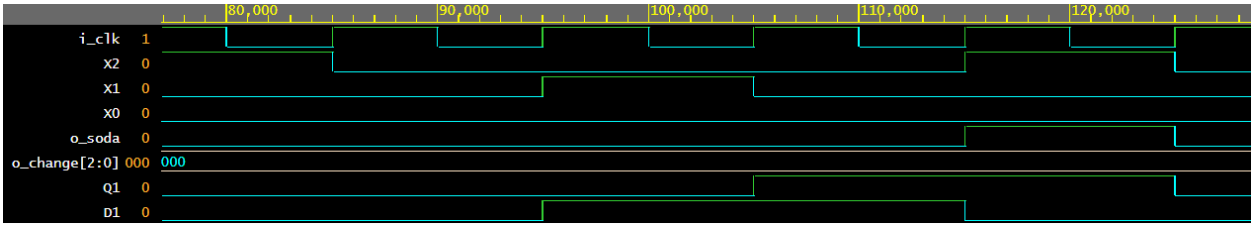
1. Trường hợp 1

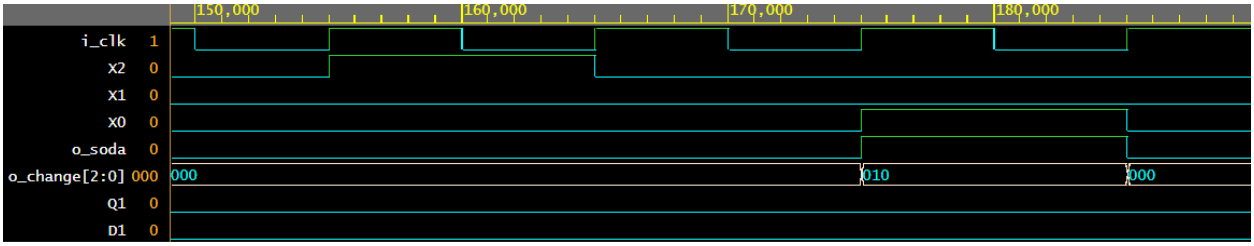
- Ban đầu bỏ 10 đồng, đợi 1 chu kỳ rồi tiếp tục bỏ 25 đồng, lúc này máy sẽ trả 1 lon soda → o_soda = 1 và trả lại 20 đồng thừa → o_change = 011



2. Trường hợp 2

- Ban đầu bỏ 5 đồng, sau đó bỏ 10 đồng, sau đó tiếp tục bỏ 5 đồng, lúc này vừa đủ 20 đồng nên máy sẽ trả 1 lon soda và không có tiền thừa → o_change = 000



3. Trường hợp 3

- Bỏ vào 5 đồng, sau đó bỏ 25 đồng, lúc này máy sẽ trả 1 lon soda và 10 đồng tiền thừa → o_change = 010